BÀI 08
NOTIFICATION và FILE

A. MỤC TIÊU:

- Trình bày cách quản lý các notification trong ứng dụng.
- Trình bày cách Flutter làm việc với file hệ thống.

B. DỤNG CỤ - THIẾT BỊ THỰC HÀNH CHO MỘT SV:

STT	Chủng loại – Quy cách vật tư	Số lượng	Đơn vị	Ghi chú
1	Computer	1	1	

C. NỘI DUNG THỰC HÀNH

1. Tóm tắt lý thuyết

1.1. Notification trong Flutter

Trong các ứng dụng di động, notification (thông báo) là một tính năng rất quen thuộc và đóng vai trò quan trọng trong việc giữ kết nối giữa ứng dụng và người dùng. Khi người dùng không mở ứng dụng, thông báo giúp ứng dụng có thể "giao tiếp" bằng cách gửi các lời nhắc, cảnh báo, hoặc thông tin mới đến đúng thời điểm. Hiện nay, rất nhiều ứng dụng phổ biến đều sử dụng notification để nâng cao hiệu quả. Ví dụ: Facebook, Zalo, Instagram dùng để báo tin nhắn mới; Shopee, Tiki, Lazada gửi thông báo về đơn hàng và khuyến mãi; Google Calendar, Todoist nhắc lịch học, công việc; MB Bank, Vietcombank thông báo giao dịch tài chính.

❖ **Các loại Notification trong Flutter**

- **Local Notification**: thông báo cục bộ, được tạo trực tiếp trong thiết bị, sử dụng packages : **flutter_local_notifications**
- **Push Notification**: thông báo từ server gửi về, cần internet, sử dụng packages :**firebase_messaging**

Bài thực hành này sẽ sử dụng Local Notification thông báo cục bộ trong ứng dụng.

❖ **Các bước sử dụng local notification trong ứng dụng flutter :**

✓ **Bước 1** : thêm packages **flutter_local_notifications** vào **pubspec.yaml** vào **dependencies**:

```
dependencies:
  flutter:
    sdk: flutter
  flutter_local_notifications: ^19.1.0
```

- Muốn notification được hiển thị sau một thời gian chỉ định cần thêm packages **timezone** vào **pubspec.yaml**

✓ **Bước 2 :** cấp quyền trong android/app/src/main/**AndroidManifest.xml**:

```xml
<manifest xmlns:android="http://schemas.android.com/apk/res/android">
    <!-- xin cấp quyền -->
    <uses-permission android:name="android.permission.RECEIVE_BOOT_COMPLETED" />
    <uses-permission android:name="android.permission.VIBRATE" />
    <uses-permission android:name="android.permission.USE_FULL_SCREEN_INTENT" />
    <!-- NOTE: the example app requests USE_EXACT_ALARM to make it easier to run the app.
         Developers will need to check if their own app needs to use SCHEDULE_EXACT_ALARM instead -->
    <uses-permission android:name="android.permission.SCHEDULE_EXACT_ALARM" />
    <uses-permission android:name="android.permission.POST_NOTIFICATIONS" />
    <uses-permission android:name="android.permission.FOREGROUND_SERVICE" />
    <!-- NOTE: Special use was selected as it's the closest match for this example app.
         apps should specify the appropriate permission for their use cases. -->
    <uses-permission android:name="android.permission.FOREGROUND_SERVICE_SPECIAL_USE" />
    <uses-permission android:name="android.permission.WAKE_LOCK" />
```

✓ **Bước 3**: chỉ định nội dung sau giữa các thẻ **<application>** để có thể hiển thị thông báo đã lên lịch:

```xml
        <meta-data
            android:name="flutterEmbedding"
            android:value="2" />

        <receiver android:exported="false" android:name="com.dexterous.flutterlocalnotifications.ScheduledNotificationReceiver" />
        <receiver android:exported="false" android:name="com.dexterous.flutterlocalnotifications.ScheduledNotificationBootReceiver">
            <intent-filter>
                <action android:name="android.intent.action.BOOT_COMPLETED"/>
                <action android:name="android.intent.action.MY_PACKAGE_REPLACED"/>
                <action android:name="android.intent.action.QUICKBOOT_POWERON" />
                <action android:name="com.htc.intent.action.QUICKBOOT_POWERON"/>
            </intent-filter>
        </receiver>
    </application>
```

✓ **Bước 4 :** Cấu hình tập tin **android\app\build.gradle.kts**

```
compileOptions {
    sourceCompatibility = JavaVersion.VERSION_17
    targetCompatibility = JavaVersion.VERSION_17
    isCoreLibraryDesugaringEnabled = true
}
```

```
dependencies {
    coreLibraryDesugaring("com.android.tools:desugar_jdk_libs:2.1.4")
}
```

❖ **Một số phương thức thường dùng trong class *FlutterLocalNotificationsPlugin* để làm việc với** Local Notification,

- `cancel({required int id, String? tag}) → Future<void>`
  - Hủy/xóa thông báo có ID được chỉ định.
- `cancelAll() → Future<void>`
  - Hủy/xóa tất cả thông báo.
- `cancelAllPendingNotifications() → Future<void>`
  - Hủy/xóa tất cả các thông báo đang chờ xử lý.
- `getActiveNotifications() → Future<List<ActiveNotification>>`
  - Trả về danh sách các thông báo đang hoạt động được ứng dụng hiển thị mà chưa bị tắt/xóa.
- `getNotificationAppLaunchDetails()→Future<NotificationAppLaunchDetails?>`

- Trả về thông tin về việc liệu thông báo được tạo từ plugin này có được sử dụng để khởi chạy ứng dụng hay không.

- `pendingNotificationRequests() → Future<List<PendingNotificationRequest>>`
  - Trả về danh sách các thông báo đang chờ được gửi/hiển thị.
- `show({required int id, String? title, String? body, NotificationDetails? notificationDetails, String? payload}) → Future<void>`
  - Hiển thị thông báo với dữ liệu tùy chọn sẽ được gửi lại cho ứng dụng khi người dùng chạm vào thông báo.
- `zonedSchedule({required int id, required TZDateTime scheduledDate, required NotificationDetails notificationDetails, required AndroidScheduleMode androidScheduleMode, String? title, String? body, String? payload, DateTimeComponents? matchDateTimeComponents}) → Future<void>`
  - Lên lịch hiển thị thông báo vào ngày và giờ cụ thể theo múi giờ nhất định.

1.2. Làm việc với file

Làm việc với file trong Flutter là một kỹ thuật quan trọng vì nó giúp ứng dụng **lưu trữ và quản lý dữ liệu lâu dài trên thiết bị người dùng**. Thay vì dữ liệu chỉ tồn tại tạm thời trong bộ nhớ (RAM), việc sử dụng file cho phép ứng dụng ghi lại thông tin như ghi chú, danh sách công việc, cấu hình người dùng hoặc lịch hẹn và có thể truy xuất lại ngay cả khi ứng dụng đã tắt. Điều này đặc biệt cần thiết đối với các ứng dụng hoạt động offline, không phụ thuộc vào Internet. Ngoài ra, làm việc với file còn giúp tối ưu hiệu năng và giảm phụ thuộc vào cơ sở dữ liệu phức tạp trong các ứng dụng nhỏ và trung bình, đồng thời là nền tảng để phát triển các hệ thống lưu trữ nâng cao như JSON, SQLite hay đồng bộ dữ liệu với server.

Trong Flutter, việc thao tác file sử dụng thư viện Dart: ***dart:io*** và package hỗ trợ: ***path_provider***

❖ **Các bước làm việc với file:**

Thêm thư viện → lấy đường dẫn → tạo file → ghi/đọc dữ liệu → xử lý lỗi → cập nhật UI

- ***Lấy đường dẫn thông qua phương thức:*** *getApplicationDocumentsDirectory():*

```
Future<String> get _path async {
  final dir = await getApplicationDocumentsDirectory();
  return dir.path;
}
```

- ***Tạo file: File(đường dẫn đến file và tên file)***

```
Future<File> get _file async {
  final path = await _path;
  return File('$path/todo.txt');
}
```

- ***Ghi dữ liệu vào file***: sử dụng phương thức ***writeAsString()*** để ghi dữ liệu dạng chuỗi (file .txt hoặc .json) *hoặc* ***writeAsBytes()*** để ghi dữ liệu dạng nhị phân (ảnh, file, audio).

```
//ghi dữ liệu vào file
Future<void> writeData(String data) async {
  final file = await _file;
  await file.writeAsString(data);
}
```

Có thể ghi thêm dữ liệu vào file bằng cách chỉ rõ **mode: FileMode.append**:

```
//ghi thêm dữ liệu vào file đã có
  Future<void> appendData(String data) async {
    final file = await _file;
    await file.writeAsString(data, mode: FileMode.append);
  }
```

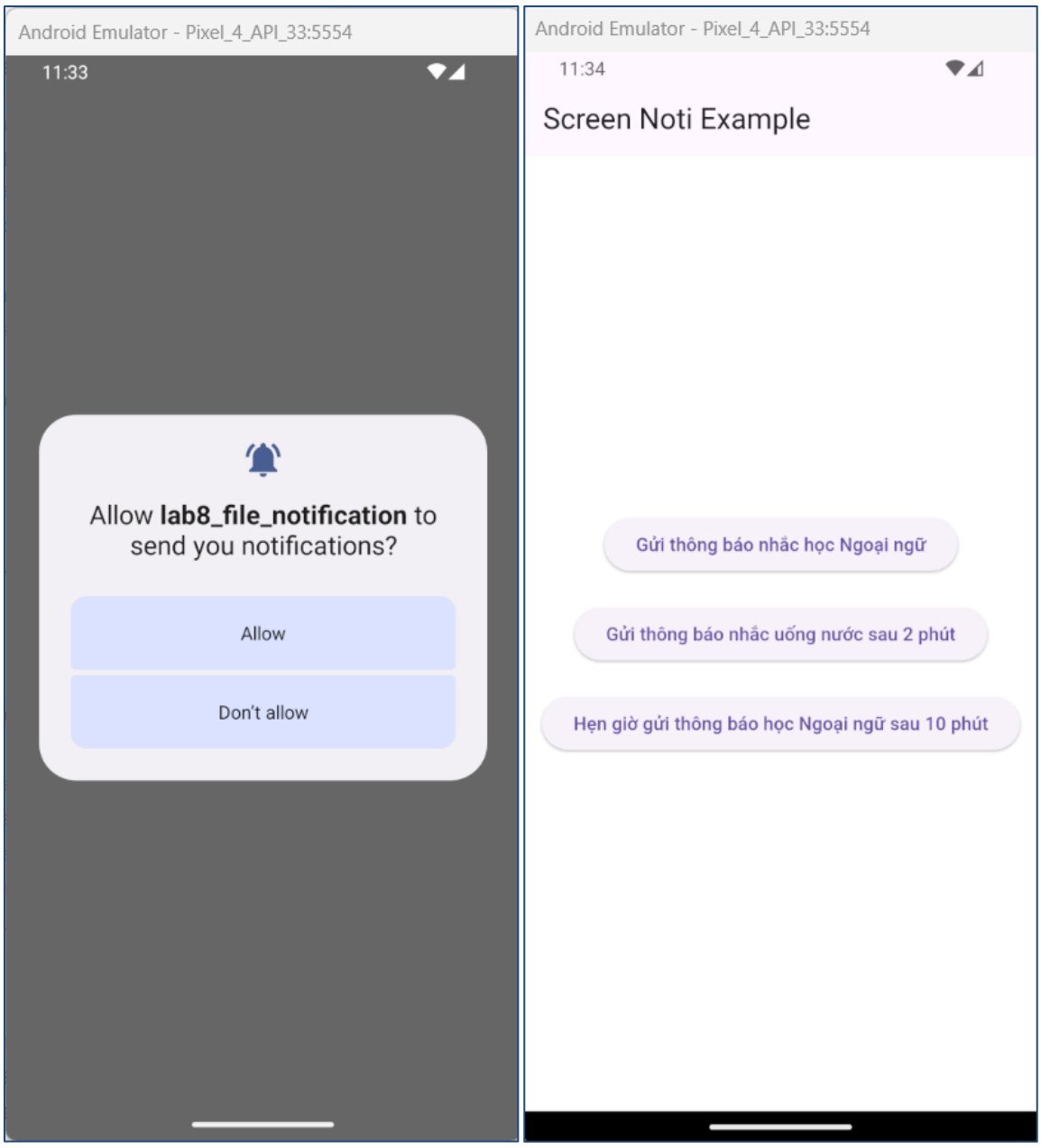
- ***Đọc dữ liệu từ file:*** sử dụng **readAsString()** hoặc **readAsBytes()**

```
//đọc dữ liệu từ file
  Future<String> readData() async {
    try {
      final file = await _file;
      return await file.readAsString();
    } catch (e) {
      return "Chưa có dữ liệu";
    }
  }
```

2. Bài tập hướng dẫn

Bài tập 1. Tạo ứng dụng có các chức năng :

- Gửi thông báo nhắc học Ngoại ngữ.
- Nhắc uống uống nước sau 2 phút.
- Hẹn giờ nhắc học Ngoại ngữ sau 10 phút.



Hướng dẫn :

❖ **Bước 1 : Cài đặt các package**

- Tạo project mới.
- Mở file **pubspec.yaml** và thêm vào **dependencies** : (lưu ý các package phải ngang cấp với flutter package)

```
dependencies:
  flutter:
    sdk: flutter
  flutter_local_notifications: ^19.1.0
  cupertino_icons: ^1.0.8
  permission_handler: ^11.3.0
  intl: ^0.18.1
  timezone: ^0.10.0
```

- **Bước 2 : Cấu hình để thực thi được ứng dụng (Bước 2, 3, 4 ở mục 1.1)**
- **Bước 3: Xây dựng class NotificationService.dart để thực hiện các yêu cầu**

Tạo file *lib/services/notification_service.dart* có nội dung như sau:

- **Khai báo sử dụng các thư viện**

```
import 'package:flutter_local_notifications/flutter_local_notifications.dart';
import 'package:timezone/data/latest.dart' as tz;
import 'package:timezone/timezone.dart' as tz;
```

- **Khởi tạo đối tượng notification**

```
class NotificationService {
  final FlutterLocalNotificationsPlugin _plugin =
      FlutterLocalNotificationsPlugin();

  // ================= INIT =================
  Future<void> init() async {
    // 1. Init timezone
    tz.initializeTimeZones();
    // FIX cứng timezone VN
    tz.setLocalLocation(tz.getLocation('Asia/Ho_Chi_Minh'));

    // 2. Android settings
    const android =
AndroidInitializationSettings('@mipmap/ic_launcher');

    const settings = InitializationSettings(android: android);

    // 3. Init plugin
    await _plugin.initialize(settings);

    // 4. Xin quyền notification
    await _requestPermission();
  }
```

- **Yêu cầu cấp quyền**

```
// ================= REQUEST PERMISSION =================
  Future<void> _requestPermission() async {
    var status = await Permission.notification.status;

    if (status.isDenied) {
      await Permission.notification.request();
    }

    if (status.isPermanentlyDenied) {
      openAppSettings();
    }
  }
```

- **Xây dựng phương thức tạo notification có thể tái sử dụng nhiều lần**

```
static AndroidNotificationDetails _createAndroidDetails({
    required String channelId,
    required String channelName,
    String? channelDescription,
  }) {
    return AndroidNotificationDetails(
      channelId,
      channelName,
      channelDescription: channelDescription,
      importance: Importance.max,
      priority: Priority.high,
      visibility: NotificationVisibility.public,
      playSound: true,
      enableVibration: true,
      showWhen: true,
    );
  }
```

- **Xây dựng phương thức hẹn lịch hiển thị thông báo**

```
// Hàm tái sử dụng cho thiết lập thời gian để bật notification
zonedSchedule
  Future<void> _scheduleNotification({
    required int id,
    required String title,
    required String body,
    required DateTime scheduledTime,
    required AndroidNotificationDetails androidDetails,
  }) async {
    await _plugin.zonedSchedule(
      id,
      title,
      body,
      tz.TZDateTime.from(scheduledTime, tz.local),
      NotificationDetails(android: androidDetails),
                                                androidScheduleMode:
AndroidScheduleMode.exactAllowWhileIdle,
      uiLocalNotificationDateInterpretation:
          UILocalNotificationDateInterpretation.absoluteTime,
    );
  }
```

- **Hàm hiển thị thông báo ngay tức thì**

```
/// Hàm gửi thông báo tức thì
  Future<void> showSimpleNotification({
    int id = 0,
    String title = 'Nhắc học từ vựng',
    String body = 'Bạn đã học 5 từ mới hôm nay chưa?',
  }) async {
    final details = _createAndroidDetails(
```

```
      channelId: 'learn_english_channel',
```

```
      channelName: 'Học Anh Văn',
    );

              await      _plugin.show(id,      title,      body,
NotificationDetails(android: details));
  }
```

- **Đặt lịch thời gian sau 1 khoảng thời gian để hiển thị thông báo**

```
/// Đặt lịch sau 1 khoảng thời gian delay
Future<void>  scheduleNotificationAfter(Duration  delay,  String
channelId, String channelName, String channelDescription) async {
  final details = _createAndroidDetails(
    channelId: channelId,
    channelName: channelName,
    channelDescription: channelDescription,
  );

  final scheduledTime = tz.TZDateTime.now(tz.local).add(delay);

  await _scheduleNotification(
    id: 1,
    title: channelName,
    body: channelDescription,
    scheduledTime: scheduledTime,
    androidDetails: details,
  );
}
```

❖ **Bước 4:** Xây dựng UI

Tạo file **lib/views/scren_noti_ex.dart** có các nội dung sau:

```
import 'package:flutter/material.dart';
import
'package:lab8_file_notification/services/notification_services.da
rt';
```

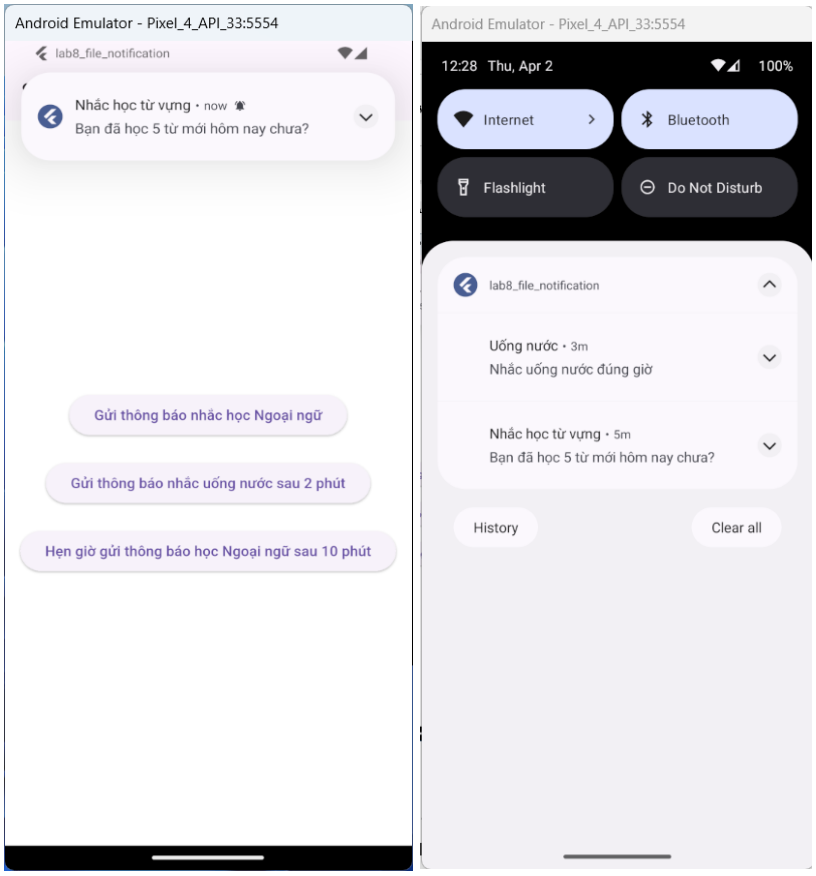
```
class ScreenNotiEx extends StatelessWidget {
  const ScreenNotiEx({super.key});
  @override
  Widget build(BuildContext context) {
             NotificationService      notificatiosnservice      =
NotificationService();
    notificatiosnservice.init();
    return Scaffold(
      appBar: AppBar(title: Text("Screen Noti Example")),
      body: Center(
        child: Column(
          mainAxisAlignment: MainAxisAlignment.center,
          children: [
            //nut lenh len lich notification
            ElevatedButton(
              onPressed: () {
                notificatiosnservice.showSimpleNotification();
              },
              child: Text("Gửi thông báo nhắc học Ngoại ngữ"),
            ),
            SizedBox(height: 20),
            ElevatedButton(
              onPressed: () {
                notificatiosnservice.scheduleNotificationAfter(
                  Duration(minutes: 2),
                  'drink_water_channel',
                  'Uống nước',
                  'Nhắc uống nước đúng giờ',
                );
              },
                child: Text("Gửi thông báo nhắc uống nước sau 2
phút"),
            ),
            SizedBox(height: 20),
```

```
            ElevatedButton(
              onPressed:
                  () {}, //sinh viên tự viết code để đặt lịch gửi thông báo nhắc học Ngoại ngữ sau 10 phút
                child: Text("Hẹn giờ gửi thông báo học Ngoại ngữ sau 10 phút"),
            ),
          ],
        ),
      ),
    );
  }
}
```

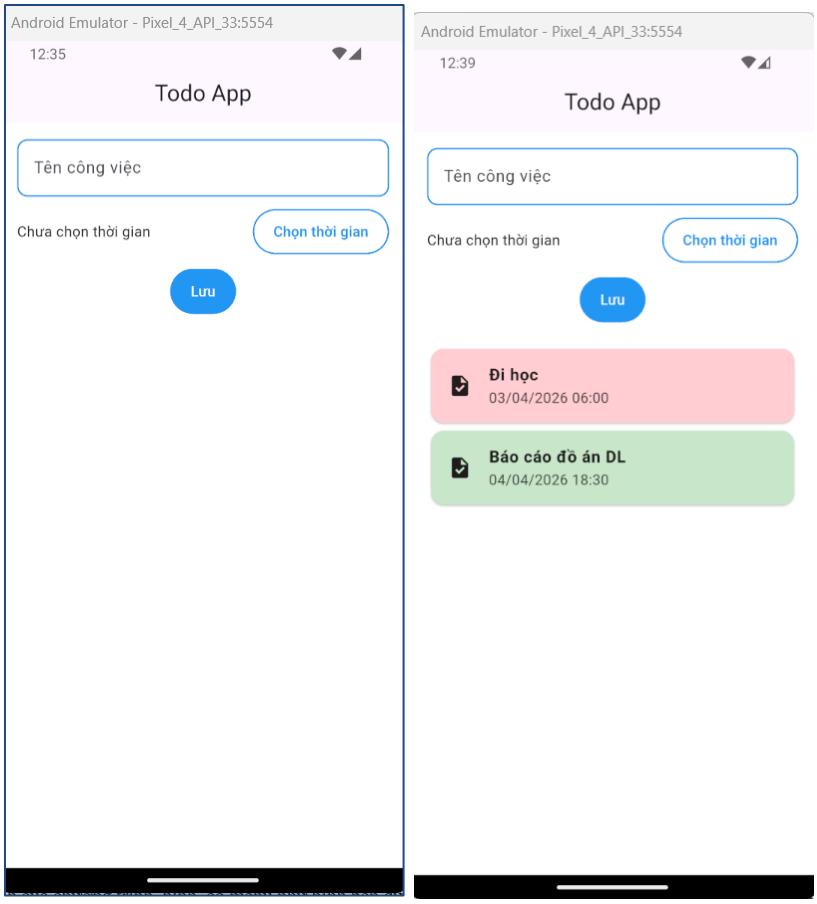
❖ **Bước 5 :** Cài đặt hàm main trong **lib\main.dart** để kích hoạt localnotification

```
void main() async {
  WidgetsFlutterBinding.ensureInitialized();
  final notificationService = NotificationService();
  await notificationService.init();
  runApp(MyApp());
}
class MyApp extends StatelessWidget {
  @override
  Widget build(BuildContext context) {
    return MaterialApp(
      debugShowCheckedModeBanner: false,
      theme: ThemeData(
         scaffoldBackgroundColor: Colors.white, // nền trắng toàn app
      ),
      home: ScreenNotiEx(),
    );
  }
}
```

Kết quả khi chạy chương trình như sau:



Bài tập 2: Hãy thiết kế ứng dụng có giao diện như sau:



Mô tả: ứng dụng cho phép người dùng nhập tên công việc, chọn thời gian thực hiện: Chọn ngày và giờ; khi nhấp "Lưu" sẽ thêm vào listview và lưu nội dung vào file "todo.txt"

Hướng dẫn:

Bước 1: thêm các package như hướng dẫn 1.2

Bước 2: tạo file ***lib/models/todo.dart*** có nội dung như sau:

```
class Todo {
  String title;
  DateTime time;
 // khởi tạo
  Todo({required this.title, required this.time});
  // convert -> string
  String toFileString() {
    return "$title|${time.toIso8601String()}";
  }
  // parse <- string
  static Todo fromString(String line) {
    final parts = line.split("|");
```

```
    return Todo(
      title: parts[0],
      time: DateTime.parse(parts[1]),
    );
  }
}
```

Bước 3: tạo file ***lib/services/file_services.dart*** có nội dung như sau:

```
import 'dart:io';
import 'package:path_provider/path_provider.dart';
import '../models/todo.dart';

class FileService {
  Future<String> get _path async {
    final dir = await getApplicationDocumentsDirectory();
    return dir.path;
  }

  Future<File> get _file async {
    final path = await _path;
    return File('$path/todo.txt');
  }

Future<void> saveTodos(List<Todo> todos) async {
    final file = await _file;
    String data = todos.map((e) => e.toFileString()).join("\n");
    await file.writeAsString(data);
  }

  // Đọc dữ liệu từ file và trả về danh sách Todo
  Future<List<Todo>> loadTodos() async {
    try {
      final file = await _file;
      final content = await file.readAsString();
```

```
      return content
          .split("\n")
          .where((e) => e.isNotEmpty)
          .map((e) => Todo.fromString(e))
          .toList();
    } catch (e) {
      return [];
    }
  }
}
```

Bước 4: tạo file lib/views/screen_todo.dart có nội dung như sau:

```
import 'package:flutter/material.dart';
import 'package:intl/intl.dart';
import '../models/todo.dart';
import '../services/file_services.dart';
class ScreenTodo extends StatefulWidget {
  @override
  State<ScreenTodo> createState() => _TodoScreenState();
}
```

```
class _TodoScreenState extends State<ScreenTodo> {
        final       TextEditingController       controller       =
TextEditingController();
  final FileService fileService = FileService();
  //khoi tao danh sach todo va thoi gian chon
  List<Todo> todos = [];
  DateTime? selectedTime;

  @override
  void initState() {
    super.initState();
    loadData();
  }
```

```
  void loadData() async {
    todos = await fileService.loadTodos();
    setState(() {});
  }

  void addTodo() async {
    if (controller.text.isEmpty || selectedTime == null) return;

        final   todo   =   Todo(title:   controller.text,   time:
selectedTime!);

    todos.add(todo);
    await fileService.saveTodos(todos);

    controller.clear();
    selectedTime = null;

    setState(() {});
  }
  // CHỌN NGÀY + GIỜ
  void pickDateTime() async {
    final date = await showDatePicker(
      context: context,
      firstDate: DateTime.now(),
      lastDate: DateTime(2100),
      initialDate: DateTime.now(),
    );

    if (date != null) {
      final time = await showTimePicker(
        context: context,
        initialTime: TimeOfDay.now(),
      );
```

```
      if (time != null) {
        selectedTime = DateTime(
          date.year,
          date.month,
          date.day,
          time.hour,
          time.minute,
        );
        setState(() {});
      }
    }
  }
  String formatTime(DateTime time) {
    return DateFormat("dd/MM/yyyy HH:mm").format(time);
  }
  @override
  Widget build(BuildContext context) {
    final List<Color> colors = [
      Colors.red.shade100,
```

```
      Colors.green.shade100,
      Colors.blue.shade100,
      Colors.yellow.shade100,
      Colors.purple.shade100,
    ];
    Color getTextColor(Color bg) {
        return bg.computeLuminance() > 0.5 ? Colors.black87 :
Colors.white;
    }
    return Scaffold(
      appBar: AppBar(title: Text("Todo App"), centerTitle: true),

      body: Padding(
        padding: EdgeInsets.all(16),
```

```
        child: Column(
          children: [
            // nhập công việc
            TextField(
              controller: controller,
              decoration: InputDecoration(
                labelText: "Tên công việc",
                filled: true,
                fillColor: Colors.white, // nền trắng
                enabledBorder: OutlineInputBorder(
                  borderSide: BorderSide(
                    color: Colors.blue,
                    width: 1.5,
                  ), // viền xanh
                  borderRadius: BorderRadius.circular(10),
                ),
                focusedBorder: OutlineInputBorder(
                  borderSide: BorderSide(
                    color: Colors.blue,
                    width: 2,
                  ), // khi click vào
                  borderRadius: BorderRadius.circular(10),
                ),

                border: OutlineInputBorder(
                  borderRadius: BorderRadius.circular(10),
                ),
              ),
            ),
            SizedBox(height: 10),
            // chọn thời gian
            Row(
              children: [
```

Expanded(

```
                  child: Text(
                    selectedTime == null
                        ? "Chưa chọn thời gian"
                        : formatTime(selectedTime!),
                  ),
                ),
                ElevatedButton(
                  onPressed: pickDateTime,
                  style: ElevatedButton.styleFrom(
                    backgroundColor: Colors.white,
                    foregroundColor: Colors.blue,
                      side: BorderSide(color: Colors.blue, width:
1.5),
                    elevation: 0, // flat modern style
                     padding: EdgeInsets.symmetric(horizontal: 20,
vertical: 12),
                    shape: RoundedRectangleBorder(
                      borderRadius: BorderRadius.circular(25),
                    ),
                  ),
                  child: Text(
                    "Chọn thời gian",
                                    style:   TextStyle(fontWeight:
FontWeight.w600),
                  ),
                ),
              ],
            ),

            SizedBox(height: 10),
            // lưu
            ElevatedButton(
              onPressed: addTodo,
```

```
              style: ElevatedButton.styleFrom(
                backgroundColor: Colors.blue,
                foregroundColor: Colors.white,
                side: BorderSide(color: Colors.blue, width: 1.5),
                elevation: 0, // flat modern style
                    padding:  EdgeInsets.symmetric(horizontal:  20,
vertical: 12),
                shape: RoundedRectangleBorder(
                  borderRadius: BorderRadius.circular(25),
                ),
              ),
              child: Text("Lưu"),
            ),
            SizedBox(height: 20),
            // hiển thị danh sách
            Expanded(
              child: ListView.builder(
                itemCount: todos.length,
                itemBuilder: (context, index) {
                  final t = todos[index];
                         final  Color  bgColor  =  colors[index  %
colors.length];
                  return Card(
                    color: bgColor, // màu nền theo thứ tự
                    child: ListTile(
                      leading: Icon(
                        Icons.task,
                        color: getTextColor(bgColor),
                      ), // icon xanh
                      title: Text(
                        t.title,
                        style: TextStyle(
                          color: getTextColor(bgColor),
```

```
                            fontWeight: FontWeight.bold,
                          ),
                        ),
                        subtitle: Text(
                          formatTime(t.time),
                          style: TextStyle(
                                                              color:
getTextColor(bgColor).withOpacity(0.7),
                          ),
                        ),
                      ),
                    );
                  },
                ),
              ),
            ],
          ),
        ),
      );
  }
}
```

Bước 5: Viết hàm main (Sinh viên tự viết)

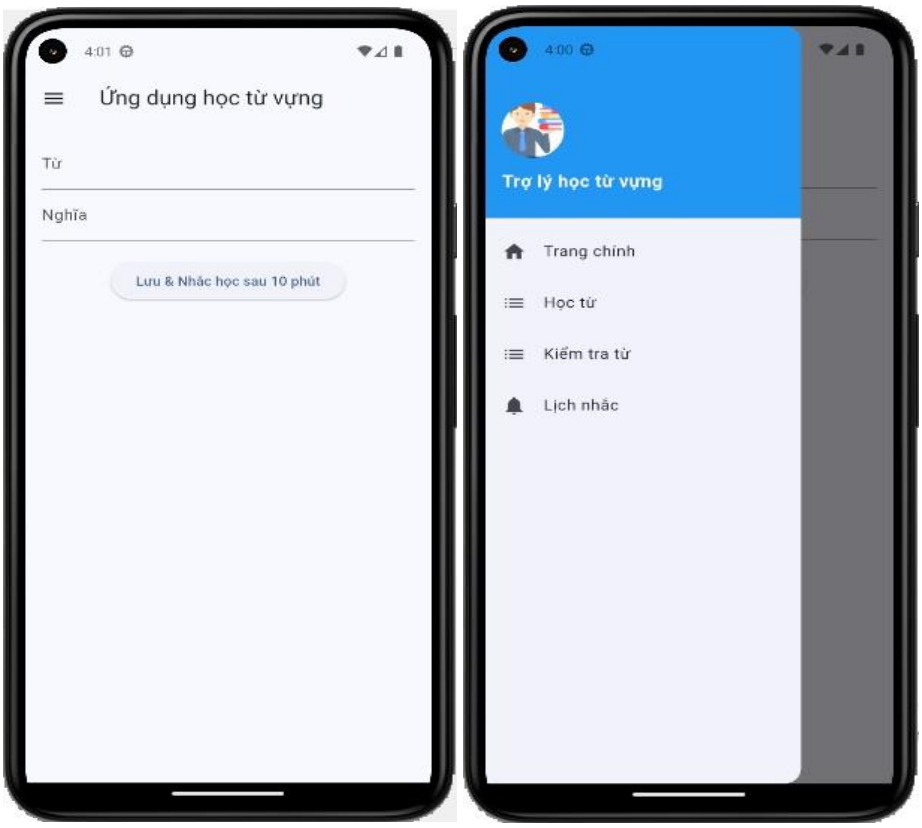
3. Bài tập tại lớp

Bài tập 3: Hãy xây dựng ứng dụng nhắc việc bằng cách sử dụng bài tập 2, bổ sung thêm các xử lý phù hợp để khi nhấp "Lưu" sẽ thực hiện thêm các hành động:

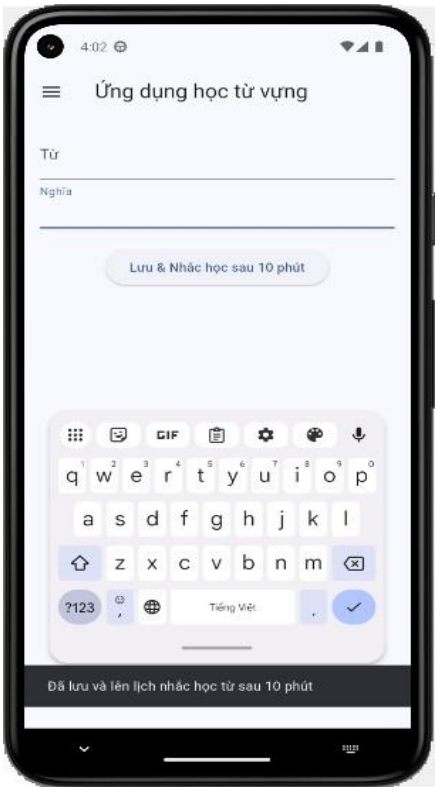
- Hiển thị notification đã thêm lịch cho công việc.
- Thiết lập lịch cho công việc theo thời gian đã chọn.

Bài tập 4: Hãy xây dựng ứng dụng hỗ trợ học tập từ vựng như sau:

Giao diện chính của chương trình hình, có menu như hình bên dưới.



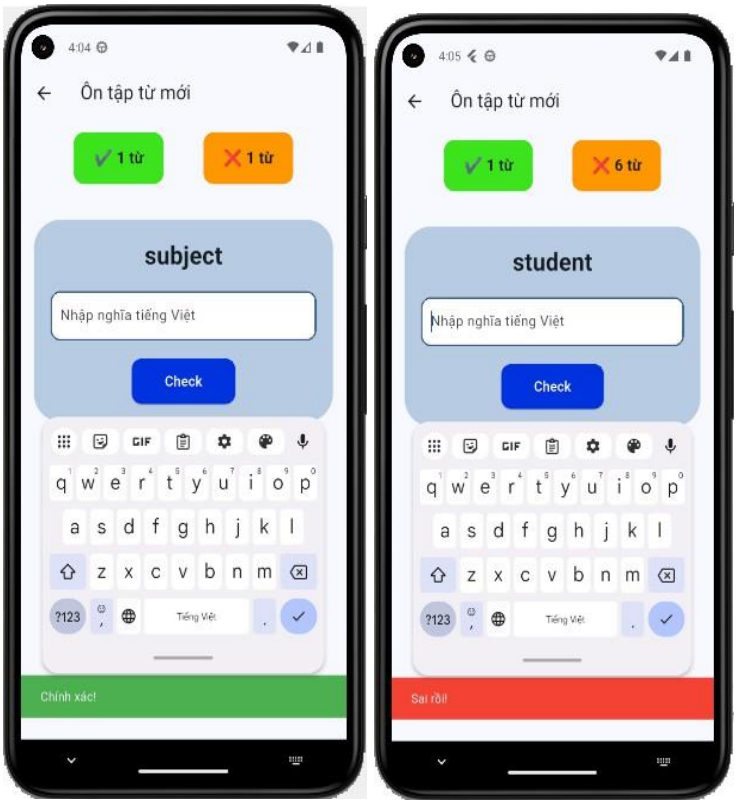
Khi nhập một từ và nghĩa của từ, chọn "Lưu và Nhắc học sau 10 phút" sẽ hiển thị một notification báo đã lưu và hãy ôn tập sau 10 phút và một thông tin ở SnackBar như hình bên dưới.



Khi chọn menu “Học từ” sẽ hiển thị giao diện như hình bên dưới:



Người dùng sẽ nhập nghĩa của từ và chọn “Check”. Ứng dụng sẽ kiểm tra từ và nghĩa đã lưu để thông báo câu trả lời đúng hay sai và cập nhật vào hai ô thông báo ở trên. Như hai màn hình minh họa bên dưới. Nếu số từ sai là bội số của 3 sẽ hiển thị một notification.



Từ màn hình chính chọn mục kiểm tra từ sẽ hiển thị màn hình sau:



Hệ thống sẽ tự hiển thị câu hỏi và đáp án. Dựa vào dữ liệu đã lưu sẽ kiểm tra đáp án đúng hay sai và cập nhật vào số câu đúng , sai. Khi hoàn thành số câu trả lời sẽ hiển thị một messagebox thông báo số câu đúng, số câu sai và tỉ lệ đúng là bao nhiêu câu.



Khi chọn "Lịch nhắc" hệ thống sẽ hiển thị danh sách các lịch được thiết lập nhắc mà chưa hiển thị notification.



4. Bài tập về nhà

Bài tập 4. Xây dựng ứng dụng todolist có giao diện như hình mẫu. Chọn biểu tượng notification để thiết lập thời gian thông báo.

Chức năng « Lưu nhiệm vụ » sẽ lưu nhiệm vụ đó vào file và trở về màn hình chính.

Khi chọn một nhiệm vụ (đánh dấu ☑) sẽ hiển thị một hộp thoại hỏi người dùng có xác nhận nhiệm vụ này đã hoàn thành ? Nếu có sẽ thông báo trên khay thông báo của hệ thống số lượng nhiệm vụ đã hoàn thành.

